CONSTRUCTION MATERIAL WORKPLACE

Construction Materials E-Commerce Web Application

PROJECT DOCUMENTATION

Prepared by: Deepak Tumma

Technology focus: React, Vite, React Router DOM, Redux Toolkit, Bootstrap/CSS

Prepared using the supplied project ZIP and the provided e-commerce project documentation reference.

Document control & scope

This document describes the Construction Material Workplace project based on the supplied source archive and the provided “Project-Based Learning Documentation and Student Outcomes” reference. It adapts the reference e-commerce architecture to the construction-material marketplace theme. It distinguishes source-tree evidence from planned or reference-design functionality; features should be marked complete only after they are tested in the running application.

Project at a glance

Item	Description
Project name	Construction Material Workplace
Project type	Frontend-focused e-commerce web application
Primary users	Customers and administrators
Frontend	React 19 with Vite
Navigation	React Router DOM
State management	Redux Toolkit and React Redux; authentication context module present
Styling	Bootstrap and CSS
Data/API approach	Reference architecture describes Axios + JSON Server (db.json); confirm runtime connection before demo

1. Introduction

Construction Material Workplace is an online shopping interface concept for construction and building materials. The application is organised around product discovery, product information, cart and wishlist actions, checkout/order views, customer account pages and administrative management screens. The intention is to make material browsing more structured and provide a single interface for common shopping tasks.

2. Problem statement

Customers looking for building supplies need a clear way to browse material types, find a product, review its information and collect chosen items before checkout. An online catalogue can make these steps easier to navigate. The project explores this workflow through a component-based React interface.

3. Objectives

• Build a clear construction-material catalogue with product and category navigation.

• Provide search and category-filtering entry points for product discovery.

• Create cart and wishlist flows with quantity and item-management actions.

• Organise customer account and order-related pages.

• Provide administrative screens for products, categories, users and orders.

• Use reusable components, route separation and shared state management to support maintainability.

4. Technology stack

Technology	Role in the project
React	Builds the interface from reusable components and pages.
Vite	Development server and production build tooling.
React Router DOM	Maps URLs to pages and supports protected-route patterns.
Redux Toolkit / React Redux	Manages shared client state; source archive includes cart, product, wishlist and order slices.
Context API	Authentication context module is present in the source tree.
Bootstrap / CSS	Provides layout utilities and application styling.
Axios + JSON Server	Described in the reference architecture for REST-style API calls and a db.json data store; verify the active connection in the supplied build.

5. System architecture

The reference project describes a flow from the user interface through React components and routing, into the Redux Toolkit store, then through API calls to JSON Server, with responses reflected in the UI. In the supplied ZIP, the source folders include pages, components, routes, context, redux and services. The actual runtime API connection should be checked before the project is presented as fully data-backed.

Layer	Responsibility
User interface	Customer and admin screens, forms, product cards and navigation.
Pages/components	Reusable visual building blocks and page-level workflows.
Routing	URL navigation and route protection patterns.
State management	Shared client state and action/reducer updates.
Service/API layer	Central place for HTTP requests.
Data server	Reference architecture uses JSON Server and db.json.

6. Functional modules

Customer experience

Home page, product catalogue, product details, category navigation, search, cart, wishlist.

Accounts & access

Register, login, logout, authentication context, protected customer routes.

Cart & orders

Quantity changes, cart totals, checkout flow, order history and status views.

Profile & support

Profile page, About, Contact and FAQ pages.

Administration

Dashboard, product create/edit/delete, user management, order status and category management.

7. Page and source organisation

The supplied source archive includes the following major areas. Names are taken from the project source structure; the exact routes and rendered behaviour should be confirmed in the running application.

• src/components: Navbar, Footer, ProductCard, SearchBar, CategoryCard and CartCard.

• src/pages: Home, Products, ProductDetails, Login, Register, Cart, Wishlist, Checkout, Orders, Profile, About, Contact and FAQ.

• src/admin: Dashboard, AdminProducts, AddProduct, EditProduct, AdminUsers, AdminOrders and AdminCategories.

• src/redux: store and slices for cart, products, wishlist and orders.

• src/context: AuthContext.jsx.

• src/services: api.js.

• src/routes: AppRoutes.jsx and ProtectedRoute.jsx.

• src/App.jsx and src/main.jsx: application entry and top-level composition.

8. User workflow

• Open the home page and browse categories or featured products.

• Search or filter the product catalogue.

• Open a product detail view to inspect product information.

• Add a selected item to the cart or wishlist.

• Review cart items and update quantities or remove items.

• Continue to checkout and review the order summary.

• Use order history/profile pages for account-related views.

• Administrators use management pages to maintain products, categories, users and order status.

9. Data design and API reference

The reference document proposes the following JSON Server collections: users, products, categories, cart, wishlist, orders, reviews, payments and contact. They represent the main entities used by a typical e-commerce interface. Confirm the data file and endpoints used by the current local setup before the demo.

Resource	Typical operations in the reference design
users	GET /users, POST /users, PUT /users/:id, DELETE /users/:id
products	GET /products, GET /products/:id, POST /products, PUT /products/:id, DELETE /products/:id
categories	GET /categories, POST /categories, PUT /categories/:id, DELETE /categories/:id
search	GET /products?q=cement (example query pattern)
cart	GET /cart, POST /cart, PUT /cart/:id, DELETE /cart/:id
wishlist	GET /wishlist, POST /wishlist, DELETE /wishlist/:id
orders	GET /orders, POST /orders, PUT /orders/:id, DELETE /orders/:id

Important implementation note

The supplied archive does not establish that every endpoint is currently connected, persistent or secure. JSON Server is suitable for a learning/demo data API, not a production commerce backend. Do not claim live payment processing or production-grade authentication unless those integrations are independently implemented and tested.

10. State management

Redux Toolkit organises shared UI state into slices. The source archive contains cartSlice.js, productSlice.js, wishlistSlice.js, orderSlice.js and store.js. Typical actions described by the reference include adding/removing cart items, increasing/decreasing quantity, clearing the cart, fetching/filtering products, managing wishlist entries and placing/fetching orders. Context API is used as the authentication/session-management pattern in the reference.

11. Authentication and route protection

The source tree contains AuthContext.jsx and ProtectedRoute.jsx, and the reference includes register, login, logout and protected routes. For a classroom demo, test both allowed and blocked navigation. For real deployment, credentials must be verified by a backend, passwords must be hashed server-side, and authorisation must not depend only on client-side route guards.

12. UI and responsiveness

The interface uses React components and CSS/Bootstrap styling. Check the layout at desktop, tablet and mobile widths. Confirm navigation, product cards, forms, empty-cart states, feedback messages and admin tables remain usable. Use genuine screenshots from the running app in a final submission if screenshots are required; none have been fabricated in this document.

13. Testing plan

Area	Test cases / expected checks
Navigation	All links and routes load; refresh on nested routes works.
Authentication	Registration, valid/invalid login, logout and protected-route checks.
Products	Catalogue loading, product detail, search, category filter, empty result.
Cart	Add/remove item, increment/decrement quantity, totals, empty-cart state.
Wishlist	Add and remove products; verify UI state is consistent.
Checkout/orders	Validate required form inputs; confirm order creation and history if API is connected.
Admin	Create/edit/delete product and category; view users; update order status.
Responsive UI	Check at common mobile and desktop widths.
API failures	Test server stopped, failed requests, loading and error feedback.

14. Setup and demonstration guide

The archive is a Vite project. Use a terminal opened in the folder containing package.json. Install dependencies if needed, start the development server, then open the local URL printed by Vite. If using JSON Server separately, start it from the folder containing the configured data file and verify that the API base URL in src/services/api.js matches the server address.

Typical frontend commands

```
npm install
npm run dev
npm run build
```

If an API server is part of the current setup, start it in a separate terminal using the command configured for that project. Do not assume the data server is included or configured merely because the reference document mentions JSON Server. Check package.json, api.js and the available data file.

Recommended live demo order

• Introduce the problem and the target users.

• Show home page and product catalogue.

• Demonstrate search/category navigation and product details.

• Demonstrate cart and wishlist interactions.

• Show login/register and explain route protection, using only verified behaviour.

• Show admin screens and demonstrate CRUD only if connected and working.

• Conclude with limitations and planned improvements.

15. Limitations

• The ZIP is primarily a frontend source project; it does not prove production deployment or a secure backend.

• JSON Server, if used, is a mock/demo data layer rather than a production database/API.

• Payment options shown in a reference design do not mean payment gateway integration is active.

• Authentication, roles, persistence, image assets and error handling must be tested in the current environment.

• Product quantity, inventory, delivery and pricing rules need backend validation for real commerce.

16. Future enhancements

• Build a secure backend API with database persistence and server-side validation.

• Implement secure authentication, role-based access and password reset.

• Integrate a payment gateway in sandbox mode before any live transactions.

• Add delivery estimates, inventory tracking, order tracking and email notifications.

• Improve product imagery, accessibility, responsive behaviour and loading/error states.

• Explore AI-assisted product recommendations and multi-vendor support after core flows are reliable.

17. Conclusion

Construction Material Workplace is an academic e-commerce project focused on construction supplies. Its React/Vite source structure separates reusable components, pages, route handling, authentication context,

Redux state and service code. The supplied reference document provides the common e-commerce feature blueprint and evaluation framework. The strongest final presentation will demonstrate only flows verified in the running application and will clearly identify any remaining work.

Appendix A — Viva questions

• What is React, and why is it suitable for a component-based interface?

• What is the difference between props and state?

• Why is React Router DOM used?

• What is a protected route?

• What are Redux Toolkit slices and the store?

• How does an action update Redux state and refresh the UI?

• What is Context API used for in authentication?

• What are GET, POST, PUT and DELETE?

• What is JSON Server, and how is it different from a production backend?

• How is the cart total calculated and how should the server validate it?

Appendix B — Reference basis

Primary project source: Construction_Material_Work_Place_Fixed.zip, supplied with this request. Reference document: “Project-Based Learning Documentation and Student Outcomes”, supplied with this request. The reference describes the general React e-commerce architecture, common modules, API patterns, folder structure, future enhancements and evaluation criteria. Project-specific statements in this report are limited to what is visible in the source archive or clearly labelled as reference design/verification guidance.

End of document